

Multivariate Polynomials

Paul Brandstetter
Matrikelnummer: k12310371

Abgabedatum: 23. April 2026

Programmieren 2

1 Quellcode

Der Code ist in 3 Files aufgeteilt.

Classen und Member Function Declaration sowie One-Liner definitionen sind in DistPoly.h

```
/*
 * DistPoly.h
 *
 * Created on: Apr 23, 2026
 * Author: luap314
 */

#include <string>

using namespace std;

class Monomial {
    int coeff = 0;
    unsigned int length = 0;
    int *exponents = nullptr;

public:

    //Constructors and destructor, copy constructor and copy assignment operator
    Monomial(int newCoeff, unsigned int newLength, int *newExponents);
    Monomial() {}
    ~Monomial() {delete[] exponents;}
    Monomial(const Monomial &other);
    Monomial& operator=(const Monomial &other);

    /*
     * static bool compare(a,b)
     * a and b need to be Monomials of the same size.
     * returns true if, lexicographically based on the exponents, a is greater than b
     * false otherwise.
     */

    static bool compare(const Monomial &a, const Monomial &b);

    /*
     * static bool sameExp(a,b)
     * a and b need to be Monomials of the same size.
     * returns true if the exponents are the same
     */
    static bool sameExp(const Monomial &a, const Monomial &b);

    /*
```

```

    * void setCoeff
    * allows non-members to change the coefficient of a Monomial
    */
    void setCoeff(int newCoeff) {
        coeff = newCoeff;
    }

    /*
    * int getCoeff
    * allows non-members to see the coefficient of a Monomial
    */
    int getCoeff() {
        return coeff;
    }

    /*
    * unsigned int getLength
    * allows non-members to see the length of a Monomial
    */
    unsigned int getLength() {
        return length;
    }

    /*
    * str(varNames)
    * varNames is an array of strings. the Length of that array needs to be
    * the same as the length of the Monomial.
    * returns a string in the form of
    *  $ax^1y^2z^3$  and so on.
    *
    * returns 0 if the length or the coefficient are 0.
    * doesn't concatenate variables with exponent 0 to the Output.
    */

    string str(string *varNames);
};

class DistPoly {

    unsigned int capacity = 10;
    unsigned int size = 0;
    unsigned int varNum = 2;
    string *varNames = nullptr;
    Monomial *terms = { };

public:

```

```

// zero-polynomial in n variables with given names
DistPoly(int n, string *vars);
// copy constructor, copy assignment operator, destructor
DistPoly(DistPoly &p);
DistPoly& operator=(const DistPoly &p);
~DistPoly();
// add new term with given coefficient and exponents to this polynomial
// and return this polynomial

/*
 * increases capacity by 2n+1.
 * needed when size gets close to capacity.
 */
void resize();

/*
 * DistPoly add(Summand)
 * adds a Monomial to the Polynomial.
 * Summand needs to be the same size as varNum
 */
DistPoly& add(Monomial Summand);

/*
 * DistPoly add(newCoeff,newExponents)
 * creates a monomial and adds it.
 * Length of newExponents needs to match Amount of Variables.
 */
DistPoly& add(int newCoeff, int *newExponents);
// add polynomial p to this polynomial and return this polynomial
DistPoly& add(DistPoly &p);
// the string representation of this polynomial
string str();

/*
 * void removeAt(index)
 * removes one Term and moves all the others accordingly.
 * index needs to be in range
 */
void removeAt(unsigned int index);

/*
 * void sanitize()
 * returns Polynomial to canonical Form.
 * Terms with 0 will be removed.
 * Reverse Lexicographical order for the terms
 * and same exponent terms get combined

```

```

    */
    void sanitize();

    /*
    * void removeZeroes()
    * Removes all Terms with coefficient 0
    */
    void removeZeroes();

    /*
    * void combineTerms()
    * combines terms with same exponent b adding their coefficients
    * only works, when reorder has been called previously.
    */
    void combineTerms(unsigned int start);

    /*
    * void reorder()
    * Orders the Terms of a polynomial in reverse lexicographical order.
    */
    void reorder();
};

```

Implementierung der Member Functions sind in DistPoly.cpp

```

/*
 * DistPoly.cpp
 *
 * Created on: Apr 23, 2026
 * Author: luap314
 */

#include "DistPoly.h"
#include <string>
#include <iostream>
#include <algorithm>
#include <stdexcept>

Monomial::Monomial(int newCoeff, unsigned int newLength, int *newExponents) {
    coeff = newCoeff;
    length = newLength;
    exponents = new int[length];

    for (unsigned int i = 0; i < newLength; i++) {
        exponents[i] = newExponents[i];
    }
}

```

```

}

bool Monomial::sameExp(const Monomial &a, const Monomial &b) {
    if (a.length != b.length) {
        return false;
    }

    for (unsigned int i = 0; i < a.length; i++) {
        if (a.exponents[i] != b.exponents[i]) {
            return false;
        }
    }
    return true;
}

bool Monomial::compare(const Monomial &a, const Monomial &b) {
    if (a.length != b.length) {
        throw std::range_error("Cannot compare Monomials of different sizes");
    }

    for (unsigned int i = 0; i < a.length; i++) {
        if (a.exponents[i] > b.exponents[i]) {
            return true;
        }
        if (a.exponents[i] < b.exponents[i]) {
            return false;
        }
    }
    return false;
}

string Monomial::str(string *varNames) {
    if (length == 0 || coeff == 0) {
        return "0";
    }
    string Output = std::to_string(coeff);
    for (unsigned int i = 0; i < length; i++) {
        if (exponents[i] != 0) {
            Output += varNames[i] + "^" + std::to_string(exponents[i]);
        }
    }
    return Output;
}

Monomial::Monomial(const Monomial &other) {
    coeff = other.coeff;

```

```

    length = other.length;
    exponents = new int[length];
    for (unsigned int i = 0; i < length; i++) {
        exponents[i] = other.exponents[i];
    }
}

Monomial& Monomial::operator=(const Monomial &other) {
    /*if (this == &other) {return *this;};
    coeff = other.coeff;
    length = other.length;

    delete[] exponents;

    exponents = new int[length];
    for (unsigned int i = 0; i < length; i++) {
        exponents[i] = other.exponents[i];
    }
    return *this;*/
    if (this == &other)
        return *this;

    delete[] exponents; // free old memory

    coeff = other.coeff;
    length = other.length;

    if (length > 0) {
        exponents = new int[length];
        for (unsigned int i = 0; i < length; i++) {
            exponents[i] = other.exponents[i];
        }
    } else {
        exponents = nullptr;
    }

    return *this;
}

DistPoly::DistPoly(int n, string *vars) {
    varNum = n;
    varNames = new string[n];
    for (int i = 0; i < n; i++) {
        varNames[i] = vars[i];
    }
    capacity = 10;
    size = 0;
}

```

```

    terms = new Monomial[capacity];
}

DistPoly::DistPoly(DistPoly &p) {
    capacity = p.capacity;
    size = p.size;
    varNum = p.varNum;

    terms = new Monomial[capacity];
    for (unsigned int i = 0; i < size; i++) {
        terms[i] = p.terms[i];
    }
    varNames = new string[varNum];
    for (unsigned int i = 0; i < varNum; i++) {
        varNames[i] = p.varNames[i];
    }
}

DistPoly& DistPoly::operator=(const DistPoly &p) {
    if (this == &p)
        return *this;

    delete[] terms;
    delete[] varNames;

    capacity = p.capacity;
    size = p.size;
    varNum = p.varNum;

    terms = new Monomial[capacity];
    for (unsigned int i = 0; i < size; i++) {
        terms[i] = p.terms[i];
    }

    varNames = new string[varNum];
    for (unsigned int i = 0; i < varNum; i++) {
        varNames[i] = p.varNames[i];
    }

    return *this;
}

DistPoly::~DistPoly() {
    delete[] terms;
    delete[] varNames;
}

```



```

void DistPoly::resize() {
    unsigned int newCapacity = 2 * capacity + 1;
    Monomial *newTerms = new Monomial[newCapacity];
    for (unsigned int i = 0; i < size; i++) {
        newTerms[i] = terms[i];
    }
    delete[] terms;
    terms = newTerms;
    capacity = newCapacity;
}

DistPoly& DistPoly::add(Monomial Summand) {
    if (varNum != Summand.getLength()) {
        throw std::range_error("Cannot add wrong-sized Monomial");
    }
    if (size == capacity) {
        resize();
    }
    terms[size] = Summand;
    size++;
    sanitize();
    return *this;
}

DistPoly& DistPoly::add(int newCoeff, int *newExponents) {
    add(Monomial(newCoeff, varNum, newExponents));
    return *this;
}

DistPoly& DistPoly::add(DistPoly &p) {
    for (unsigned int i = 0; i < p.size; i++) {
        add(p.terms[i]);
    }
    return *this;
}

string DistPoly::str() {
    if (size == 0) {
        return "0";
    }

    string Output = "";
    for (unsigned int i = 0; i < size - 1; i++) {
        Output += terms[i].str(varNames) + " + ";
    }
}

```

```

    }
    Output += terms[size - 1].str(varNames);
    return Output;
}

void DistPoly::removeAt(unsigned int index) {
    if (index >= size) {
        throw std::out_of_range("Index out of bounds");
    }

    for (unsigned int i = index; i < size - 1; i++) {
        terms[i] = terms[i + 1];
    }

    size--; // just shrink logical size
}

void DistPoly::reorder() {
    std::sort(terms, terms + size, Monomial::compare);
}

void DistPoly::combineTerms(unsigned int start) {
    //Note:: Only use when reorder has been used before
    if (start >= size - 1) {
        return;
    }
    if (Monomial::sameExp(terms[start], terms[start + 1])) {
        terms[start].setCoeff(
            terms[start].getCoeff() + terms[start + 1].getCoeff());
        removeAt(start + 1);
        combineTerms(start);
    }
    combineTerms(start + 1);
}

void DistPoly::removeZeroes() {
    unsigned int i = 0;
    while (i < size) {
        if (terms[i].getCoeff() == 0) {
            removeAt(i);
        } else {
            i++;
        }
    }
}
}

```

```

void DistPoly::sanitize() {
    reorder();
    combineTerms(0);
    removeZeroes();
}

```

int main sowie alle Tests sind in DistPolyMain.cpp

```

/*
 * DistPolyMain.cpp
 *
 * Created on: Apr 23, 2026
 * Author: luap314
 */

#include "DistPoly.h"
#include <string>
#include <iostream>
#include <random>

using namespace std;

int main() {

    // some exponent vectors ("power products")
    int e1[2] = { 1, 2 };
    int e2[2] = { 2, 1 };
    int e3[2] = { 1, 0 };
    int e4[2] = { 0, 1 };
    int e5[2] = { 0, 0 };
    int e6[2] = { 2, 2 };

    string vars2[2] = { "x", "y" };
    // construct zero polynomial in two variables, then add monomials

    DistPoly p(2, vars2);
    p.add(3, e1).add(5, e2).add(7, e3).add(11, e4).add(13, e5);
    // construct zero polynomial in two variables, then add monomials
    DistPoly q(2, vars2);
    q.add(11, e4).add(-3, e2).add(2, e6).add(-2, e2);
    // print p and q
    cout << p.str() << endl;
    cout << q.str() << endl;
}

```

```

// set p to p+2*q and print it
DistPoly r = p;
r.add(q).add(q);
p = r;
cout << p.str() << endl;

//My Own Tests

//rng stuff
default_random_engine rng;
random_device rand_dev;
rng.seed(rand_dev());
uniform_int_distribution<int> next_r(0, 10);

//Single Var

string vars1[1] = { "x" };

int a1[1] = { next_r(rng) };
int a2[1] = { next_r(rng) };
int a3[1] = { next_r(rng) };
int a4[1] = { next_r(rng) };

DistPoly Test(1, vars1);
DistPoly Test1(1, vars1);

Test.add(Test1);

//0 Polynomial
cout << Test.str() << endl;

Test.add(next_r(rng), a1).add(next_r(rng), a2).add(next_r(rng), a3).add(
    next_r(rng), a4);

cout << Test.str() << endl;

//Will this output 0?

Test1.add(5, a2).add(-5, a2);
cout << Test1.str() << endl;

string vars3[3] = { "x", "y", "z" };

int b1[3] = { next_r(rng), next_r(rng), next_r(rng) };
int b2[3] = { next_r(rng), next_r(rng), next_r(rng) };

```

```

DistPoly Test3(3, vars3);

//It adds from left to right

Test3.add(1, b1).add(Test3).add(4, b2).add(Test3);

//Therefore we should have one coefficient 4 and one 8;

cout << Test3.str() << endl;

try {
    DistPoly n(3, vars3);
    n.add(3, e1);
    n.add(Test1);
} catch (const std::exception &e) {
    std::cout << "Error: " << e.what() << std::endl;
}
return 0;
}

```

2 Tests

Es wurde nicht mit Inputs getestet. Output des Codes war:

```
5x^2y^1 + 3x^1y^2 + 7x^1 + 11y^1 + 13
2x^2y^2 + -5x^2y^1 + 11y^1
4x^2y^2 + -5x^2y^1 + 3x^1y^2 + 7x^1 + 33y^1 + 13
0
7x^10 + 1x^5 + 2x^4
0
4x^3y^7z^4 + 8y^2z^10
Error: Cannot add wrong-sized Monomial
```

Das Programm hat mit exit value 0 terminiert. Alle Outputs sind Erwartungsgemäß.